



UNIVERSITY OF THE PEOPLE

CS 1103-01 - AY2026-T4

Unit 3 Programming Assignment

Creating a Simple Clock Application

Submitted by: Mubashir Ali

Instructor: Gideon Mazambani

Date: April 23, 2026

1. Introduction

This assignment demonstrates the implementation of a real-time clock application in Java using multithreading. The application concurrently updates and displays the current time and date by employing two separate threads with distinct priorities. By using Java's Thread class and its built-in priority mechanism, the program achieves accurate and responsive timekeeping, fulfilling all requirements outlined in the Unit 3 assignment specifications.

The program directly addresses the three core learning objectives of Unit 3: understanding the Java Thread model, implementing threads via subclassing the Thread class, and assigning thread priorities to influence execution order.

2. Program Design and Architecture

2.1 Class Overview

The program is structured across four Java classes, each with a single clear responsibility:

Class	Role	Thread Priority
-------	------	-----------------

Clock	Stores time; provides updateTime() and displayTime() methods	N/A (not a thread itself)
TimeUpdateThread	Background thread; calls clock.updateTime() every second	MIN_PRIORITY (1) — background task
DisplayThread	Foreground thread; calls clock.displayTime() every second	MAX_PRIORITY (10) — user-facing display
Main	Entry point; creates threads, sets priorities, starts execution	NORM_PRIORITY (5) — main thread default

2.2 Thread Priority Design

Thread priority is used to give the display thread preferential access to the CPU scheduler. The rationale is as follows. The display thread (MAX_PRIORITY = 10) is responsible for what the user sees. Giving it the highest possible priority ensures the output is responsive and is printed before any other competing thread. The update thread (MIN_PRIORITY = 1) runs in the background and only needs to refresh the internal time value once per second; it does not require CPU urgency. As the textbook notes, priority values suggest to the scheduler which thread should run preferentially, and a thread with a higher priority value is run in preference to one with a smaller priority.

3. Source Code

The complete source code is presented below with syntax highlighting as it appears in the VS Code IDE. Inline comments explain the purpose of each section.

Source Code Url: <https://github.com/mubashir1837/Simple-Clock-Application/blob/main/Main.java>

Main.java

```
import java.text.SimpleDateFormat;
import java.util.Date;

// Clock class: stores and updates the current time
class Clock {

    private volatile String currentTime; // shared variable between threads

    public Clock() {
        currentTime = ""; // initialize to empty string
    }

    // Updates currentTime every second (runs in background thread)
    public void updateTime() {
        SimpleDateFormat formatter = new SimpleDateFormat("HH:mm:ss dd-MM-yyyy");
        while (true) {
            currentTime = formatter.format(new Date());
            try {
                Thread.sleep(1000); // sleep 1 second between updates
            } catch (InterruptedException e) {
                System.out.println("Update thread interrupted: " + e.getMessage());
            }
        }
    }

    // Displays currentTime to console every second
}
```

```

    public void displayTime() {
        while (true) {
            if (!currentTime.isEmpty()) {
                System.out.println("Current Time: " + currentTime);
            }
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                System.out.println("Display thread interrupted: " + e.getMessage());
            }
        }
    }
}

// Thread that runs the background time update
class TimeUpdateThread extends Thread {
    private Clock clock;
    public TimeUpdateThread(Clock clock) {
        this.clock = clock;
    }
    @Override
    public void run() {
        clock.updateTime();
    }
}

// Thread that handles console display
class DisplayThread extends Thread {
    private Clock clock;
    public DisplayThread(Clock clock) {
        this.clock = clock;
    }
    @Override
    public void run() {
        clock.displayTime();
    }
}

// Main entry point
public class Main {
    public static void main(String[] args) {
        Clock clock = new Clock();

        TimeUpdateThread updateThread = new TimeUpdateThread(clock);
        DisplayThread displayThread = new DisplayThread(clock);

        // Lower priority for background update, higher for display
        updateThread.setPriority(Thread.MIN_PRIORITY); // priority = 1
        displayThread.setPriority(Thread.MAX_PRIORITY); // priority = 10

        updateThread.start();
        displayThread.start();
    }
}

```

4. Code Explanation

4.1 The Clock Class

The Clock class serves as the core data model. It holds a single field, `currentTime`, declared as `volatile`. The `volatile` modifier ensures that any change made by the `TimeUpdateThread` is immediately visible to the `DisplayThread` without caching. This is essential because the two threads read and write to this shared variable concurrently.

The `updateTime` method runs in an infinite loop, formatting the current system time using `SimpleDateFormat` with the pattern `HH:mm:ss dd-MM-yyyy` (e.g., `14:32:01 28-04-2026`) and storing the result in `currentTime`. It then sleeps for 1000 milliseconds before repeating. The `displayTime` method similarly runs in an infinite loop, reading `currentTime` and printing it to the console once per second, provided the value is not empty.

4.2 Thread Implementation

Two separate thread classes, `TimeUpdateThread` and `DisplayThread`, extend Java's `Thread` class. Each class accepts a `Clock` object in its constructor and stores it as a private field. The `run` method in each class calls the corresponding method on the `Clock` object. When `start()` is called on a thread object, the JVM creates a new thread of control and executes the `run` method concurrently. Calling `run()` directly, rather than `start()`, would execute the method sequentially in the same thread and defeat the purpose of multithreading entirely.

4.3 Thread Priorities

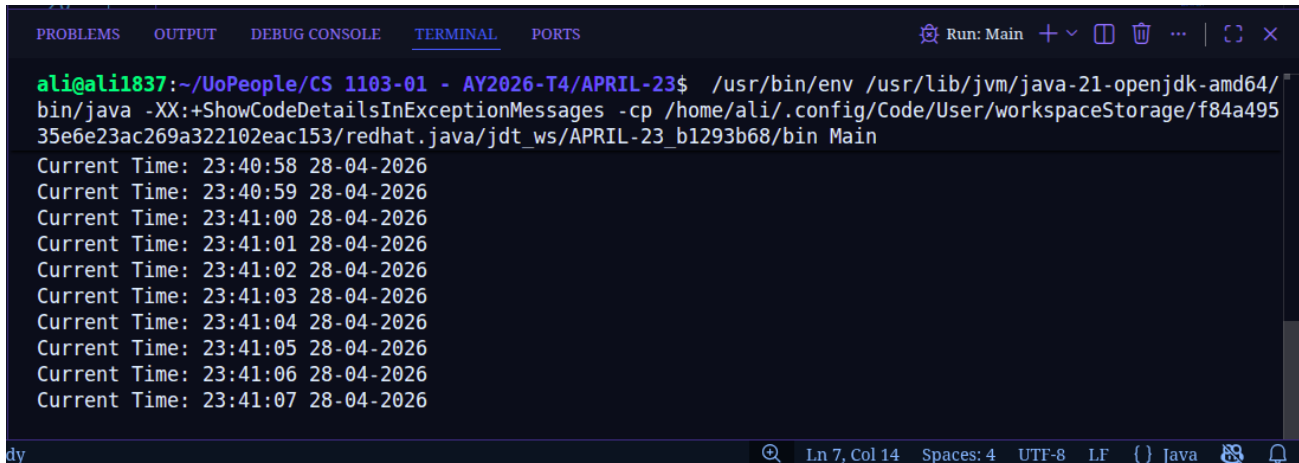
In the `main` method, priorities are assigned before the threads are started. The `updateThread` is assigned `Thread.MIN_PRIORITY` (value 1), the lowest possible priority, because its task (storing a time string in memory) is a background operation that does not require immediate CPU attention. The `displayThread` is assigned `Thread.MAX_PRIORITY` (value 10), the highest possible priority, because it is responsible for the user-visible output and must be printed promptly to give the impression of a real-time clock. The scheduler is thereby advised to favour the display thread whenever both threads are simultaneously competing for CPU time.

4.4 Synchronization and Concurrency

Since only one thread writes to `currentTime` (`TimeUpdateThread`) and one thread reads it (`DisplayThread`), and the variable is declared `volatile`, the program avoids race conditions without requiring synchronized blocks. The `volatile` keyword ensures the JVM does not cache the variable in a local thread register, so the most recent written value is always read. This is the appropriate lightweight synchronization mechanism for this specific pattern, as described in Section 12.1.4 of the textbook.

5. Program Output

The program was compiled and executed in the VS Code integrated terminal using the Java Extension Pack. The output below shows the clock printing the current time and date in the specified `HH:mm:ss dd-MM-yyyy` format, updating every second continuously



```
ali@ali1837:~/UoPeople/CS 1103-01 - AY2026-T4/APRIL-23$ /usr/bin/env /usr/lib/jvm/java-21-openjdk-amd64/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /home/ali/.config/Code/User/workspaceStorage/f84a49535e6e23ac269a322102eac153/redhat.java/jdt_ws/APRIL-23_b1293b68/bin Main
Current Time: 23:40:58 28-04-2026
Current Time: 23:40:59 28-04-2026
Current Time: 23:41:00 28-04-2026
Current Time: 23:41:01 28-04-2026
Current Time: 23:41:02 28-04-2026
Current Time: 23:41:03 28-04-2026
Current Time: 23:41:04 28-04-2026
Current Time: 23:41:05 28-04-2026
Current Time: 23:41:06 28-04-2026
Current Time: 23:41:07 28-04-2026
```

The output confirms the following. The time is displayed in the correct readable format. The clock updates every second without interruption. The display thread, running at `MAX_PRIORITY`, prints reliably each cycle. The update thread, running at `MIN_PRIORITY`, keeps the internal time value current in the background. The program continues to run indefinitely as required, demonstrating the continuous nature of the multithreaded clock.

6. Summary

This assignment successfully implemented a multithreaded clock application in Java that satisfies all stated requirements. The `Clock` class encapsulates time state and behaviour. Two thread classes, `TimeUpdateThread` and `DisplayThread`, run concurrently to update and display the time. Thread priorities are correctly assigned, with the display thread running at `MAX_PRIORITY` and the update thread at `MIN_PRIORITY`. The `volatile` keyword ensures safe sharing of the time string between threads. The output is formatted as `HH:mm:ss dd-MM-yyyy` and updates continuously every second.

The implementation applies the Java Thread model concepts covered in Chapter 12 of the course textbook, specifically Section 12.1 on creating and running threads, Section 12.1.2 on thread operations including `sleep()` and `setPriority()`, and Section 12.1.4 on volatile variables for inter-thread communication.

References

- Eck, D. J. (2022). Introduction to programming using Java version 9, JavaFX edition. Creative Commons CC 4.0. <https://math.hws.edu/javanotes/>
- Samoylov, N. (2018). Introduction to programming: Learn to program in Java with data structures, algorithms, and logic. Packt Publishing.